In [10]:
```
# %pip install pandas numpy matplotlib seaborn scikit-learn scipy openpyxl p
```

In [11]:
```
# %pip install opencv-python
```

In [ ]:
```
# 1)
02
# 2)
# Write a program to implement Huffman Encoding using a greedy strategy
```

In [157…
```
import heapq
```

In [158…
```
class Node:
    def __init__(self,char,freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None
```

In [164…
```
def get_codes(root, code, codes):
    if root is None:
        return
    if root.char is not None:
        codes[root.char] = code or "0"
        return

    get_codes(root.left, code + "0", codes)
    get_codes(root.right, code + "1", codes)
```

In [165…
```
#huffman coding
def hufffman(text):
    if not text:
        return {}, ""

    freq ={}
    for char in text:
        freq[char] = freq.get(char,0) + 1


    heap = []
    count =0

    for char,f in freq.items():
        heapq.heappush(heap,(f,count, Node(char,f)))
        count = count +1

    while len(heap) >1:
        f1,_,left = heapq.heappop(heap)
        f2,_,right= heap.heappop(heap)

        parent = Node(None, f1+f2)
        parent.left = left
        parent.right = right
```

```
        heapq.heappush(heap, parent.freq,count,parent)
        count = count+1

    codes = {}
    get_codes(heap[0][2], "", codes)

    encoded = ""
    for char in text:
        encoded = encoded + codes[char]

    return codes , encoded


text = input("enteer text:")

codes, encoded = huffman(text)

print("Huffman codes: ", codes)
print("Encoded text: ", encoded)
```

```
Huffman codes:  {'k': '00', 'w': '010', 'n': '011', 't': '100', 'i': '101',
'a': '11'}
Encoded text:  11010110111001010011
```

In [ ]: